

React component code:

```
import React from 'react';
```

```
const Tooltip = () => {
  return (
    <div className="group relative flex justify-center items-center text-zinc-600 text-sm font-bold">
      <div className="absolute opacity-0 group-hover:opacity-100 group-hover:-translate-y-[150%] -
translate-y-[300%] duration-500 group-hover:delay-500 skew-y-[20deg] group-hover:skew-y-0
shadow-md">
        <div className="bg-lime-200 flex items-center gap-1 p-2 rounded-md">
          <svg fill="none" viewBox="0 0 24 24" height="20px" width="20px"
xmlns="http://www.w3.org/2000/svg" className="stroke-zinc-600">
            <circle strokeLinejoin="round" r={9} cy={12} cx={12} />
            <path strokeLinejoin="round" d="M12 3C12 3 8.5 6 8.5 12C8.5 18 12 21 12 21" />
            <path strokeLinejoin="round" d="M12 3C12 3 15.5 6 15.5 12C15.5 18 12 21 12 21" />
            <path strokeLinejoin="round" d="M3 12H21" />
            <path strokeLinejoin="round" d="M19.5 7.5H4.5" />
            <g filter="url(#filter0_d_15_556)">
              <path strokeLinejoin="round" d="M19.5 16.5H4.5" />
            </g>
            <defs>
              <filter colorInterpolationFilters="sRGB" filterUnits="userSpaceOnUse" height={3} width={17}
y={16} x="3.5" id="filter0_d_15_556">
                <feFlood result="BackgroundImageFix" floodOpacity={0} />
                <feColorMatrix result="hardAlpha" values="0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 127 0"
type="matrix" in="SourceAlpha" />
                <feOffset dy={1} />
                <feGaussianBlur stdDeviation="0.5" />
                <feColorMatrix values="0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0.1 0" type="matrix" />
                <feBlend result="effect1_dropShadow_15_556" in2="BackgroundImageFix" mode="normal"
/>
                <feBlend result="shape" in2="effect1_dropShadow_15_556" in="SourceGraphic"
mode="normal" />
              </filter>
            </defs>
          </svg>
          <span>Uiverse.io</span>
        </div>
        <div className="shadow-md bg-lime-200 absolute bottom-0 translate-y-1/2 left-1/2 translate-x-
full rotate-45 p-1" />
          <div className="rounded-md bg-white group-hover:opacity-0 group-hover:scale-[115%] group-
hover:delay-700 duration-500 w-full h-full absolute top-0 left-0">
            <div className="border-b border-r border-white bg-white absolute bottom-0 translate-y-1/2
left-1/2 translate-x-full rotate-45 p-1" />
          </div>
          <div className="shadow-md flex items-center group-hover:gap-2 bg-gradient-to-br from-lime-
200 to-yellow-200 p-3 rounded-full cursor-pointer duration-300">
            <svg fill="none" viewBox="0 0 24 24" height="20px" width="20px"
xmlns="http://www.w3.org/2000/svg" className="fill-zinc-600">
              <path strokeLinejoin="round" strokeLinecap="round" d="M15.4306 7.70172C7.55045 7.99826
3.43929 15.232 2.17021 19.3956C2.07701 19.7014 2.31139 20 2.63107 20C2.82491 20 3.0008
19.8828 3.08334 19.7074C6.04179 13.4211 12.7066 12.3152 15.514 12.5639C15.7583 12.5856
15.9333 12.7956 15.9333 13.0409V15.1247C15.9333 15.5667 16.4648 15.7913 16.7818
15.4833L20.6976 11.6784C20.8723 11.5087 20.8993 11.2378 20.7615 11.037L16.8456
5.32965C16.5677 4.92457 15.9333 5.12126 15.9333 5.61253V7.19231C15.9333 7.46845 15.7065
```

```

7.69133 15.4306 7.70172Z" /></svg><span className="text-[0px] group-hover:text-sm duration-
300">Check Our Website</span>
</div>
</div>
);
}

```

export default Tooltip;

Use loading bar for a small time

import React from 'react';

import styled from 'styled-components';

```

const Loader = () => {
  return (
    <StyledWrapper>
      <div className="loading-wave">
        <div className="loading-bar" />
        <div className="loading-bar" />
        <div className="loading-bar" />
        <div className="loading-bar" />
      </div>
    </StyledWrapper>
  );
}

```

const StyledWrapper = styled.div`

```

.loading-wave {
  width: 300px;
  height: 100px;
  display: flex;
  justify-content: center;
  align-items: flex-end;
}

```

```

.loading-bar {
  width: 20px;
  height: 10px;
  margin: 0 5px;
  background-color: #3498db;
  border-radius: 5px;
  animation: loading-wave-animation 1s ease-in-out infinite;
}

```

```

.loading-bar:nth-child(2) {
  animation-delay: 0.1s;
}

```

```

.loading-bar:nth-child(3) {
  animation-delay: 0.2s;
}

```

```

.loading-bar:nth-child(4) {
  animation-delay: 0.3s;
}

```

@keyframes loading-wave-animation {

```

0% {
  height: 10px;
}

50% {
  height: 50px;
}

100% {
  height: 10px;
}
};

```

export default Loader;

Blur text ## Integrate the <BlurText /> component from React Bits

You are helping integrate an open-source React component into an existing application.

Component: BlurText
 ### Variant: JavaScript + Tailwind
 ### Dependencies: motion

Usage Example

```

```jsx
import BlurText from "./BlurText";

const handleAnimationComplete = () => {
 console.log('Animation completed!');
};

<BlurText
 text="Isn't this so cool?!"
 delay={150}
 animateBy="words"
 direction="top"
 onAnimationComplete={handleAnimationComplete}
 className="text-2xl mb-8"
/>
```

```

Props

| Prop | Type | Default | Description |
|---------------------|----------|-----------|--|
| text | string | "" | The text content to animate. |
| animateBy | string | "words" | Determines whether to animate by 'words' or 'letters'. |
| direction | string | "top" | Direction from which the words/letters appear ('top' or 'bottom'). |
| delay | number | 200 | Delay between animations for each word/letter (in ms). |
| stepDuration | number | 0.35 | The time taken for each letter/word to animate (in seconds). |
| threshold | number | 0.1 | Intersection threshold for triggering the animation. |
| rootMargin | string | "0px" | Root margin for the intersection observer. |
| onAnimationComplete | function | undefined | Callback function triggered when all animations complete. |

Full Component Source

```

``jsx
import { motion } from 'motion/react';
import { useEffect, useRef, useState, useMemo } from 'react';

const buildKeyframes = (from, steps) => {
  const keys = new Set([...Object.keys(from), ...steps.flatMap(s => Object.keys(s))]);

  const keyframes = {};
  keys.forEach(k => {
    keyframes[k] = [from[k], ...steps.map(s => s[k])];
  });
  return keyframes;
};

const BlurText = ({
  text = "",
  delay = 200,
  className = "",
  animateBy = 'words',
  direction = 'top',
  threshold = 0.1,
  rootMargin = '0px',
  animationFrom,
  animationTo,
  easing = t => t,
  onAnimationComplete,
  stepDuration = 0.35
}) => {
  const elements = animateBy === 'words' ? text.split(' ') : text.split("");
  const [inView, setInView] = useState(false);
  const ref = useRef(null);

  useEffect(() => {
    if (!ref.current) return;
    const observer = new IntersectionObserver(
      ([entry]) => {
        if (entry.isIntersecting) {
          setInView(true);
          observer.unobserve(ref.current);
        }
      },
      { threshold, rootMargin }
    );
    observer.observe(ref.current);
    return () => observer.disconnect();
  }, [threshold, rootMargin]);

  const defaultFrom = useMemo(
    () =>
      direction === 'top' ? { filter: 'blur(10px)', opacity: 0, y: -50 } : { filter: 'blur(10px)', opacity: 0, y: 50 },
    [direction]
  );

  const defaultTo = useMemo(
    () => [
      {
        filter: 'blur(5px)',

```

```

      opacity: 0.5,
      y: direction === 'top' ? 5 : -5
    },
    { filter: 'blur(0px)', opacity: 1, y: 0 }
  ],
  [direction]
);

const fromSnapshot = animationFrom ?? defaultFrom;
const toSnapshots = animationTo ?? defaultTo;

const stepCount = toSnapshots.length + 1;
const totalDuration = stepDuration * (stepCount - 1);
const times = Array.from({ length: stepCount }, (_, i) => (stepCount === 1 ? 0 : i / (stepCount - 1)));

return (
  <p ref={ref} className={`blur-text ${className} flex flex-wrap`} >
    {elements.map((segment, index) => {
      const animateKeyframes = buildKeyframes(fromSnapshot, toSnapshots);

      const spanTransition = {
        duration: totalDuration,
        times,
        delay: (index * delay) / 1000
      };
      spanTransition.ease = easing;

      return (
        <motion.span
          className="inline-block will-change-[transform,filter,opacity]"
          key={index}
          initial={fromSnapshot}
          animate={inView ? animateKeyframes : fromSnapshot}
          transition={spanTransition}
          onAnimationComplete={index === elements.length - 1 ? onAnimationComplete : undefined}
        >
          {segment === ' ' ? '\u00A0' : segment}
          {animateBy === 'words' && index < elements.length - 1 && '\u00A0'}
        </motion.span>
      );
    })}
  </p>
);
};

```

export default BlurText;

...

Integration Instructions

1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import and render the component using the usage example above as a starting point.
4. Adjust props as needed for the specific use case — refer to the props table for all available options.

Gradual blur

Integrate the <GradualBlur /> component from React Bits

You are helping integrate an open-source React component into an existing application.

Component: GradualBlur
Variant: JavaScript + Tailwind
Dependencies: mathjs

Usage Example

```
``jsx
// Component added by Ansh - github.com/ansh-dhanani

import GradualBlur from './GradualBlur';

<section style={{position: 'relative',height: 500,overflow: 'hidden'}}>
  <div style={{ height: '100%',overflowY: 'auto',padding: '6rem 2rem' }}>
    <!-- Content Here - such as an image or text -->
  </div>

  <GradualBlur
    target="parent"
    position="bottom"
    height="6rem"
    strength={2}
    divCount={5}
    curve="bezier"
    exponential={true}
    opacity={1}
  />
</section>
``
```

Props

Prop	Type	Default	Description
position	string	"bottom"	Edge to attach the blur overlay.
strength	number	2	Base blur strength multiplier (affects each layer).
height	string	"6rem"	Overlay height (for top / bottom positions).
width	string	—	Custom width (optional). Defaults to 100% for vertical positions or matches height for horizontal positions.
divCount	number	5	Number of stacked blur layers (higher = smoother gradient).
exponential	boolean	false	Use exponential progression for stronger end blur.
curve	string	"bezier"	Distribution curve applied to layer progression.
opacity	number	1	Opacity applied to each blur layer.
animated	boolean	false	Fade in (true) or reveal on scroll ("scroll").
duration	string	"0.3s"	Animation duration (when animated).
easing	string	"ease-out"	Animation easing (opacity / backdrop-filter).
hoverIntensity	number	—	Multiplier applied to strength while hovered.
target	string	"parent"	Position relative to parent container or the entire page (fixed).
preset	string	—	Apply a predefined configuration bundle.
responsive	boolean	false	Enable internal responsive recalculation (experimental).
zIndex	number	1000	Base z-index (page target adds +100).
onAnimationComplete	() => void	—	Callback fired after animated reveal completes.
className	string	—	Additional class names appended to root element.
style	React.CSSProperties	—	Inline style overrides merged into container style.

Full Component Source

```
``jsx
import React, { useEffect, useRef, useState, useMemo } from 'react';

const DEFAULT_CONFIG = {
  position: 'bottom',
  strength: 2,
  height: '6rem',
  divCount: 5,
  exponential: false,
  zIndex: 1000,
  animated: false,
  duration: '0.3s',
  easing: 'ease-out',
  opacity: 1,
  curve: 'linear',
  responsive: false,
  target: 'parent',
  className: '',
  style: {}
};

const PRESETS = {
  top: { position: 'top', height: '6rem' },
  bottom: { position: 'bottom', height: '6rem' },
  left: { position: 'left', height: '6rem' },
  right: { position: 'right', height: '6rem' },
  subtle: { height: '4rem', strength: 1, opacity: 0.8, divCount: 3 },
  intense: { height: '10rem', strength: 4, divCount: 8, exponential: true },
  smooth: { height: '8rem', curve: 'bezier', divCount: 10 },
  sharp: { height: '5rem', curve: 'linear', divCount: 4 },
  header: { position: 'top', height: '8rem', curve: 'ease-out' },
  footer: { position: 'bottom', height: '8rem', curve: 'ease-out' },
  sidebar: { position: 'left', height: '6rem', strength: 2.5 },
  'page-header': { position: 'top', height: '10rem', target: 'page', strength: 3 },
  'page-footer': { position: 'bottom', height: '10rem', target: 'page', strength: 3 }
};

const CURVE_FUNCTIONS = {
  linear: p => p,
  bezier: p => p * p * (3 - 2 * p),
  'ease-in': p => p * p,
  'ease-out': p => 1 - Math.pow(1 - p, 2),
  'ease-in-out': p => (p < 0.5 ? 2 * p * p : 1 - Math.pow(-2 * p + 2, 2) / 2)
};

const mergeConfigs = (...configs) => configs.reduce((acc, c) => ({ ...acc, ...c }), {});

const getGradientDirection = position => {
  const directions = {
    top: 'to top',
    bottom: 'to bottom',
    left: 'to left',
    right: 'to right'
  };
  return directions[position] || 'to bottom';
}
```

```

};

const debounce = (fn, wait) => {
  let t;
  return (...a) => {
    clearTimeout(t);
    t = setTimeout(() => fn(...a), wait);
  };
};

const useResponsiveDimension = (responsive, config, key) => {
  const [val, setVal] = useState(config[key]);
  useEffect(() => {
    if (!responsive) return;
    const calc = () => {
      const w = window.innerWidth;
      let v = config[key];
      if (w <= 480 && config['mobile' + key[0].toUpperCase() + key.slice(1)])
        v = config['mobile' + key[0].toUpperCase() + key.slice(1)];
      else if (w <= 768 && config['tablet' + key[0].toUpperCase() + key.slice(1)])
        v = config['tablet' + key[0].toUpperCase() + key.slice(1)];
      else if (w <= 1024 && config['desktop' + key[0].toUpperCase() + key.slice(1)])
        v = config['desktop' + key[0].toUpperCase() + key.slice(1)];
      setVal(v);
    };
    const deb = debounce(calc, 100);
    calc();
    window.addEventListener('resize', deb);
    return () => window.removeEventListener('resize', deb);
  }, [responsive, config, key]);
  return responsive ? val : config[key];
};

const useIntersectionObserver = (ref, shouldObserve = false) => {
  const [isVisible, setIsVisible] = useState(!shouldObserve);

  useEffect(() => {
    if (!shouldObserve || !ref.current) return;

    const observer = new IntersectionObserver(([entry]) => setIsVisible(entry.isIntersecting), { threshold: 0.1 });

    observer.observe(ref.current);
    return () => observer.disconnect();
  }, [ref, shouldObserve]);

  return isVisible;
};

const GradualBlur = props => {
  const containerRef = useRef(null);
  const [isHovered, setIsHovered] = useState(false);

  const config = useMemo(() => {
    const presetConfig = props.preset && PRESETS[props.preset] ? PRESETS[props.preset] : {};
    return mergeConfigs(DEFAULT_CONFIG, presetConfig, props);
  }, [props]);

```



```

const responsiveHeight = useResponsiveDimension(config.responsive, config, 'height');
const responsiveWidth = useResponsiveDimension(config.responsive, config, 'width');
const isVisible = useIntersectionObserver(containerRef, config.activated === 'scroll');

const blurDivs = useMemo(() => {
  const divs = [];
  const increment = 100 / config.divCount;
  const currentStrength =
    isHovered && config.hoverIntensity ? config.strength * config.hoverIntensity : config.strength;

  const curveFunc = CURVE_FUNCTIONS[config.curve] || CURVE_FUNCTIONS.linear;

  for (let i = 1; i <= config.divCount; i++) {
    let progress = i / config.divCount;
    progress = curveFunc(progress);

    let blurValue;
    if (config.exponential) {
      blurValue = Math.pow(2, progress * 4) * 0.0625 * currentStrength;
    } else {
      blurValue = 0.0625 * (progress * config.divCount + 1) * currentStrength;
    }
    const p1 = Math.round((increment * i - increment) * 10) / 10;
    const p2 = Math.round(increment * i * 10) / 10;
    const p3 = Math.round((increment * i + increment) * 10) / 10;
    const p4 = Math.round((increment * i + increment * 2) * 10) / 10;
    let gradient = `transparent ${p1}%, black ${p2}%`;
    if (p3 <= 100) gradient += `, black ${p3}%`;
    if (p4 <= 100) gradient += `, transparent ${p4}%`;

    const direction = getGradientDirection(config.position);

    const divStyle = {
      position: 'absolute',
      inset: '0',
      maskImage: `linear-gradient(${direction}, ${gradient})`,
      WebkitMaskImage: `linear-gradient(${direction}, ${gradient})`,
      backdropFilter: `blur(${blurValue.toFixed(3)}rem)`,
      WebkitBackdropFilter: `blur(${blurValue.toFixed(3)}rem)`,
      opacity: config.opacity,
      transition:
        config.activated && config.activated !== 'scroll'
          ? `backdrop-filter ${config.duration} ${config.easing}`
          : undefined
    };

    divs.push(<div key={i} style={divStyle} />);
  }

  return divs;
}, [config, isHovered]);

const containerStyle = useMemo(() => {
  const isVertical = ['top', 'bottom'].includes(config.position);
  const isHorizontal = ['left', 'right'].includes(config.position);
  const isPageTarget = config.target === 'page';

```

```

const baseStyle = {
  position: isPageTarget ? 'fixed' : 'absolute',
  pointerEvents: config.hoverIntensity ? 'auto' : 'none',
  opacity: isVisible ? 1 : 0,
  transition: config.animated ? `opacity ${config.duration} ${config.easing}` : undefined,
  zIndex: isPageTarget ? config.zIndex + 100 : config.zIndex,
  ...config.style
};

if (isVertical) {
  baseStyle.height = responsiveHeight;
  baseStyle.width = responsiveWidth || '100%';
  baseStyle[config.position] = 0;
  baseStyle.left = 0;
  baseStyle.right = 0;
} else if (isHorizontal) {
  baseStyle.width = responsiveWidth || responsiveHeight;
  baseStyle.height = '100%';
  baseStyle[config.position] = 0;
  baseStyle.top = 0;
  baseStyle.bottom = 0;
}

return baseStyle;
}, [config, responsiveHeight, responsiveWidth, isVisible]);

const { hoverIntensity, animated, onAnimationComplete, duration } = config;
useEffect(() => {
  if (isVisible && animated === 'scroll' && onAnimationComplete) {
    const t = setTimeout(() => onAnimationComplete(), parseFloat(duration) * 1000);
    return () => clearTimeout(t);
  }
}, [isVisible, animated, onAnimationComplete, duration]);

return (
  <div
    ref={containerRef}
    className={`gradual-blur ${config.target === 'page' ? 'gradual-blur-page' : 'gradual-blur-parent'}
    ${config.className}`}
    style={containerStyle}
    onMouseEnter={hoverIntensity ? () => setIsHovered(true) : undefined}
    onMouseLeave={hoverIntensity ? () => setIsHovered(false) : undefined}
  >
    <div className="gradual-blur-inner relative w-full h-full">{blurDivs}</div>
  </div>
);
};

const GradualBlurMemo = React.memo(GradualBlur);
GradualBlurMemo.displayName = 'GradualBlur';
GradualBlurMemo.PRESETS = PRESETS;
GradualBlurMemo.CURVE_FUNCTIONS = CURVE_FUNCTIONS;
export default GradualBlurMemo;
...

```

Integration Instructions

1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import and render the component using the usage example above as a starting point.
4. Adjust props as needed for the specific use case — refer to the props table for all available options.

Shape blur for cards

Integrate the <ShapeBlur /> component from React Bits

You are helping integrate an open-source React component into an existing application.

Component: ShapeBlur

Variant: JavaScript + Tailwind

Dependencies: three

Usage Example

```
```jsx
```

```
import ShapeBlur from './ShapeBlur';
```

```
<div style={{position: 'relative', height: '500px', overflow: 'hidden'}}>
```

```
 <ShapeBlur
```

```
 variation={0}
```

```
 pixelRatioProp={window.devicePixelRatio || 1}
```

```
 shapeSize={0.5}
```

```
 roundness={0.5}
```

```
 borderSize={0.05}
```

```
 circleSize={0.5}
```

```
 circleEdge={1}
```

```
 />
```

```
</div>
```

```
```
```

Props

| Prop | Type | Default | Description |
|------|------|---------|-------------|
|------|------|---------|-------------|

| | | | |
|-----------|--------|---|---|
| variation | number | 0 | Selects the shape variation (0-3) used by the shader. |
|-----------|--------|---|---|

| | | | |
|----------------|--------|---|---|
| pixelRatioProp | number | 2 | Overrides the pixel ratio, typically set to the device pixel ratio. |
|----------------|--------|---|---|

| | | | |
|-----------|--------|-----|---------------------------------|
| shapeSize | number | 1.2 | Controls the size of the shape. |
|-----------|--------|-----|---------------------------------|

| | | | |
|-----------|--------|-----|--|
| roundness | number | 0.4 | Determines the roundness of the shape's corners. |
|-----------|--------|-----|--|

| | | | |
|------------|--------|------|-----------------------------------|
| borderSize | number | 0.05 | Sets the thickness of the border. |
|------------|--------|------|-----------------------------------|

| | | | |
|------------|--------|-----|---|
| circleSize | number | 0.3 | Determines the size of the hover circle effect. |
|------------|--------|-----|---|

| | | | |
|------------|--------|-----|---|
| circleEdge | number | 0.5 | Controls the edge softness of the hover circle. |
|------------|--------|-----|---|

Full Component Source

```
```jsx
```

```
import { useEffect, useRef } from 'react';
```

```
import * as THREE from 'three';
```

```
const vertexShader = /* glsl */ `
```

```
varying vec2 v_texcoord;
```

```
void main() {
```

```
 gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
```

```
 v_texcoord = uv;
```

```
}
```

```
`;
```

```

const fragmentShader = /* glsl */ `
varying vec2 v_texcoord;

uniform vec2 u_mouse;
uniform vec2 u_resolution;
uniform float u_pixelRatio;

uniform float u_shapeSize;
uniform float u_roundness;
uniform float u_borderSize;
uniform float u_circleSize;
uniform float u_circleEdge;

#ifndef PI
#define PI 3.1415926535897932384626433832795
#endif
#ifndef TWO_PI
#define TWO_PI 6.2831853071795864769252867665590
#endif

#ifndef VAR
#define VAR 0
#endif

#ifndef FNC_COORD
#define FNC_COORD
vec2 coord(in vec2 p) {
 p = p / u_resolution.xy;
 if (u_resolution.x > u_resolution.y) {
 p.x *= u_resolution.x / u_resolution.y;
 p.x += (u_resolution.y - u_resolution.x) / u_resolution.y / 2.0;
 } else {
 p.y *= u_resolution.y / u_resolution.x;
 p.y += (u_resolution.x - u_resolution.y) / u_resolution.x / 2.0;
 }
 p -= 0.5;
 p *= vec2(-1.0, 1.0);
 return p;
}
#endif

#define st0 coord(gl_FragCoord.xy)
#define mx coord(u_mouse * u_pixelRatio)

float sdRoundRect(vec2 p, vec2 b, float r) {
 vec2 d = abs(p - 0.5) * 4.2 - b + vec2(r);
 return min(max(d.x, d.y), 0.0) + length(max(d, 0.0)) - r;
}

float sdCircle(in vec2 st, in vec2 center) {
 return length(st - center) * 2.0;
}

float sdPoly(in vec2 p, in float w, in int sides) {
 float a = atan(p.x, p.y) + PI;
 float r = TWO_PI / float(sides);
 float d = cos(floor(0.5 + a / r) * r - a) * length(max(abs(p) * 1.0, 0.0));
 return d * 2.0 - w;
}

```

```

float aastep(float threshold, float value) {
 float afwidth = length(vec2(dFdx(value), dFdy(value))) * 0.70710678118654757;
 return smoothstep(threshold - afwidth, threshold + afwidth, value);
}
float fill(in float x) { return 1.0 - aastep(0.0, x); }
float fill(float x, float size, float edge) {
 return 1.0 - smoothstep(size - edge, size + edge, x);
}
float stroke(in float d, in float t) { return (1.0 - aastep(t, abs(d))); }
float stroke(float x, float size, float w, float edge) {
 float d = smoothstep(size - edge, size + edge, x + w * 0.5) - smoothstep(size - edge, size + edge, x - w
* 0.5);
 return clamp(d, 0.0, 1.0);
}

float strokeAA(float x, float size, float w, float edge) {
 float afwidth = length(vec2(dFdx(x), dFdy(x))) * 0.70710678;
 float d = smoothstep(size - edge - afwidth, size + edge + afwidth, x + w * 0.5)
 - smoothstep(size - edge - afwidth, size + edge + afwidth, x - w * 0.5);
 return clamp(d, 0.0, 1.0);
}

void main() {
 vec2 st = st0 + 0.5;
 vec2 posMouse = mx * vec2(1., -1.) + 0.5;

 float size = u_shapeSize;
 float roundness = u_roundness;
 float borderSize = u_borderSize;
 float circleSize = u_circleSize;
 float circleEdge = u_circleEdge;

 float sdfCircle = fill(
 sdCircle(st, posMouse),
 circleSize,
 circleEdge
);

 float sdf;
 if (VAR == 0) {
 sdf = sdRoundRect(st, vec2(size), roundness);
 sdf = strokeAA(sdf, 0.0, borderSize, sdfCircle) * 4.0;
 } else if (VAR == 1) {
 sdf = sdCircle(st, vec2(0.5));
 sdf = fill(sdf, 0.6, sdfCircle) * 1.2;
 } else if (VAR == 2) {
 sdf = sdCircle(st, vec2(0.5));
 sdf = strokeAA(sdf, 0.58, 0.02, sdfCircle) * 4.0;
 } else if (VAR == 3) {
 sdf = sdPoly(st - vec2(0.5, 0.45), 0.3, 3);
 sdf = fill(sdf, 0.05, sdfCircle) * 1.4;
 }

 vec3 color = vec3(1.0);
 float alpha = sdf;
 gl_FragColor = vec4(color.rgb, alpha);
}

```

```

}
`;

const ShapeBlur = ({
 className = "",
 variation = 0,
 pixelRatioProp = 2,
 shapeSize = 1.2,
 roundness = 0.4,
 borderSize = 0.05,
 circleSize = 0.3,
 circleEdge = 0.5
}) => {
 const mountRef = useRef();

 useEffect(() => {
 const mount = mountRef.current;
 if (!mount) return;

 let active = true;
 let animationFrameId;
 let time = 0,
 lastTime = 0;

 const vMouse = new THREE.Vector2();
 const vMouseDamp = new THREE.Vector2();
 const vResolution = new THREE.Vector2();

 let w = 1,
 h = 1;

 const scene = new THREE.Scene();
 const camera = new THREE.OrthographicCamera();
 camera.position.z = 1;

 const renderer = new THREE.WebGLRenderer({ alpha: true });
 renderer.setClearColor(0x000000, 0);
 mount.appendChild(renderer.domElement);

 const geo = new THREE.PlaneGeometry(1, 1);
 const material = new THREE.ShaderMaterial({
 vertexShader,
 fragmentShader,
 uniforms: {
 u_mouse: { value: vMouseDamp },
 u_resolution: { value: vResolution },
 u_pixelRatio: { value: pixelRatioProp },
 u_shapeSize: { value: shapeSize },
 u_roundness: { value: roundness },
 u_borderSize: { value: borderSize },
 u_circleSize: { value: circleSize },
 u_circleEdge: { value: circleEdge }
 },
 defines: { VAR: variation },
 transparent: true
 });
 });

```

```

const quad = new THREE.Mesh(geo, material);
scene.add(quad);

const onPointerMove = e => {
 const rect = mount.getBoundingClientRect();
 vMouse.set(e.clientX - rect.left, e.clientY - rect.top);
};

document.addEventListener('mousemove', onPointerMove);
document.addEventListener('pointermove', onPointerMove);

const resize = () => {
 if (!active) return;
 w = mount.clientWidth;
 h = mount.clientHeight;
 const dpr = Math.min(window.devicePixelRatio, 2);

 renderer.setSize(w, h);
 renderer.setPixelRatio(dpr);

 camera.left = -w / 2;
 camera.right = w / 2;
 camera.top = h / 2;
 camera.bottom = -h / 2;
 camera.updateProjectionMatrix();

 quad.scale.set(w, h, 1);
 vResolution.set(w, h).multiplyScalar(dpr);
 material.uniforms.u_pixelRatio.value = dpr;
};

resize();
window.addEventListener('resize', resize);

const ro = new ResizeObserver(() => {
 if (!active) return;
 resize();
});
ro.observe(mount);

const update = () => {
 if (!active) return;
 time = performance.now() * 0.001;
 const dt = time - lastTime;
 lastTime = time;

 ['x', 'y'].forEach(k => {
 vMouseDamp[k] = THREE.MathUtils.damp(vMouseDamp[k], vMouse[k], 8, dt);
 });

 renderer.render(scene, camera);
 animationFrameId = requestAnimationFrame(update);
};
update();

return () => {
 active = false;

```

```

 cancelAnimationFrame(animationFrameId);
 window.removeEventListener('resize', resize);
 ro.disconnect();
 document.removeEventListener('mousemove', onPointerMove);
 document.removeEventListener('pointermove', onPointerMove);
 if (mount.contains(rendererer.domElement)) {
 mount.removeChild(rendererer.domElement);
 }
 rendererer.dispose();
 rendererer.forceContextLoss();
 };
}, [variation, pixelRatioProp, shapeSize, roundness, borderSize, circleSize, circleEdge]);

return <div ref={mountRef} className={`w-full h-full ${className}`} />;
};

```

export default ShapeBlur;

...

### ### Integration Instructions

1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import and render the component using the usage example above as a starting point.
4. Adjust props as needed for the specific use case — refer to the props table for all available options.

Star border

## Integrate the <StarBorder /> component from React Bits

You are helping integrate an open-source React component into an existing application.

### Component: StarBorder

### Variant: JavaScript + Tailwind

---

### ### Usage Example

```

```jsx
import StarBorder from './StarBorder'

```

```

<StarBorder
  as="button"
  className="custom-class"
  color="cyan"
  speed="5s"
>
  // content
</StarBorder>
```

```

### ### Props

| Prop      | Type   | Default | Description                                                        |
|-----------|--------|---------|--------------------------------------------------------------------|
| as        | string | button  | Allows specifying the type of the parent component to be rendered. |
| className | string | -       | Allows adding custom classes to the component.                     |



| color | string | white | Changes the main color of the border (fades to transparent) |  
| speed | string | 6s | Changes the speed of the animation. |  
| thickness | number | 3 | Controls the thickness of the star border effect. |

### ### Full Component Source

```
```jsx
const StarBorder = ({
  as: Component = 'button',
  className = '',
  color = 'white',
  speed = '6s',
  thickness = 1,
  children,
  ...rest
}) => {
  return (
    <Component
      className={`relative inline-block overflow-hidden rounded-[20px] ${className}`}
      style={{
        padding: `${thickness}px 0`,
        ...rest.style
      }}
      {...rest}
    >
      <div
        className="absolute w-[300%] h-[50%] opacity-70 bottom-[-11px] right-[-250%] rounded-full
        animate-star-movement-bottom z-0"
        style={{
          background: `radial-gradient(circle, ${color}, transparent 10%)`,
          animationDuration: speed
        }}
      ></div>
      <div
        className="absolute w-[300%] h-[50%] opacity-70 top-[-10px] left-[-250%] rounded-full
        animate-star-movement-top z-0"
        style={{
          background: `radial-gradient(circle, ${color}, transparent 10%)`,
          animationDuration: speed
        }}
      ></div>
      <div className="relative z-1 bg-gradient-to-b from-black to-gray-900 border border-gray-800
        text-white text-center text-[16px] py-[16px] px-[26px] rounded-[20px]">
        {children}
      </div>
    </Component>
  );
};
```

export default StarBorder;

```
// tailwind.config.js
// module.exports = {
//   theme: {
//     extend: {
//       animation: {
//         'star-movement-bottom': 'star-movement-bottom linear infinite alternate',
//         'star-movement-top': 'star-movement-top linear infinite alternate',
//       }
//     }
//   }
// }
```

```
//   },
//   keyframes: {
//     'star-movement-bottom': {
//       '0%': { transform: 'translate(0%, 0%)', opacity: '1' },
//       '100%': { transform: 'translate(-100%, 0%)', opacity: '0' },
//     },
//     'star-movement-top': {
//       '0%': { transform: 'translate(0%, 0%)', opacity: '1' },
//       '100%': { transform: 'translate(100%, 0%)', opacity: '0' },
//     },
//   },
// },
// },
// }
// }
```

...

Integration Instructions

1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import and render the component using the usage example above as a starting point.
4. Adjust props as needed for the specific use case — refer to the props table for all available options.

Spotlight card

Integrate the `<SpotlightCard />` component from React Bits

You are helping integrate an open-source React component into an existing application.

Component: SpotlightCard

Variant: JavaScript + Tailwind

Usage Example

```
```jsx
import SpotlightCard from './SpotlightCard';

<SpotlightCard className="custom-spotlight-card" spotlightColor="rgba(0, 229, 255, 0.2)">
 // Content goes here
</SpotlightCard>
```
```

Props

| Prop | Type | Default | Description |
|----------------|--------|---------------------------|--|
| spotlightColor | string | rgba(255, 255, 255, 0.25) | Controls the color of the radial gradient used for the spotlight effect. |
| className | string | — | Allows adding custom classes to the component. |

Full Component Source

```
```jsx
import { useRef, useState } from 'react';

const SpotlightCard = ({ children, className = '', spotlightColor = 'rgba(255, 255, 255, 0.25)' }) => {
 const divRef = useRef(null);
 const [isFocused, setIsFocused] = useState(false);
```

```

const [position, setPosition] = useState({ x: 0, y: 0 });
const [opacity, setOpacity] = useState(0);

const handleMouseMove = e => {
 if (!divRef.current || !isFocused) return;

 const rect = divRef.current.getBoundingClientRect();
 setPosition({ x: e.clientX - rect.left, y: e.clientY - rect.top });
};

const handleFocus = () => {
 setIsFocused(true);
 setOpacity(0.6);
};

const handleBlur = () => {
 setIsFocused(false);
 setOpacity(0);
};

const handleMouseEnter = () => {
 setOpacity(0.6);
};

const handleMouseLeave = () => {
 setOpacity(0);
};

return (
 <div
 ref={divRef}
 onMouseMove={handleMouseMove}
 onFocus={handleFocus}
 onBlur={handleBlur}
 onMouseEnter={handleMouseEnter}
 onMouseLeave={handleMouseLeave}
 className={`relative rounded-3xl border border-neutral-800 bg-neutral-900 overflow-hidden p-8
 ${className}`}
 >
 <div
 className="pointer-events-none absolute inset-0 opacity-0 transition-opacity duration-500
 ease-in-out"
 style={{
 opacity,
 background: `radial-gradient(circle at ${position.x}px ${position.y}px, ${spotlightColor},
 transparent 80%)`
 }}
 />
 {children}
 </div>
);
};

export default SpotlightCard;

```

### ### Integration Instructions

1. Install any listed dependencies.
2. Copy the component source into the appropriate directory in the project.
3. Import and render the component using the usage example above as a starting point.
4. Adjust props as needed for the specific use case — refer to the props table for all available options.